Quantum Spiking Neural Network for Network Intrusion Detection

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Network intrusion detection systems (NIDS) have to classify large volumes of traffic quickly while keeping false alarms low. Quantum machine learning (QML) and spiking neural networks (SNNs) have both been suggested for building small, cheap detectors, yet in intrusion detection they have only been studied on their own. This paper presents QSNN-NIDS, a hybrid network in which every neuron is a quantum leaky integrate-and-fire (QLIF) unit: a parameterized quantum circuit produces the input current, and a classical leaky membrane turns that current into binary spikes over time. Flow records are phase-coded over 12 time steps and processed by two QLIF layers of 8 qubits each. We evaluate the model on UNSW-NB15 and CIC-IDS2017 against three classical baselines (Random Forest, RBF-SVM, MLP) and two quantum baselines (a ZZ-kernel QSVC and a QCNN), all trained on the same features and splits. QSNN-NIDS reaches an F1 score of 0.863 and ROC-AUC of 0.938 on UNSW-NB15, and an F1 of 0.937 and ROC-AUC of 0.994 on CIC-IDS2017, with 340 trainable parameters (96 of them quantum). It is competitive with the quantum baselines but trails the classical ones on both datasets. Under simulated depolarizing noise, the model keeps at least 97% of its noise-free F1 up to p = 0.02 on both datasets; at higher noise it fails in a way that depends on the dataset. Neurons fire on 14.4% to 18.0% of time steps. An independent re-run on a larger UNSW-NB15 sample reproduces the initial results to within 0.01 on accuracy, recall, MCC and ROC-AUC. All results are from a single seed.**

***Index Terms*—Quantum machine learning, spiking neural networks, network intrusion detection, variational quantum circuits, noise robustness, UNSW-NB15, CIC-IDS2017**

I. Introduction

Networks now carry far more traffic, from far more devices, than manual monitoring can handle. Intrusion detection systems watch this traffic and raise alerts when a flow looks malicious, and machine learning has become the standard way to build them [1]. In practice a detector must catch most attacks without burying analysts in false alarms, and it must keep pace with live traffic. Deep networks are good at the first part. They tend to be large and power hungry, though, and that makes them awkward to run on gateways and edge devices.

Two lines of research look at the cost side of this problem. One is quantum machine learning. Variational quantum circuits and quantum kernels map data into very high dimensional Hilbert spaces using only a handful of qubits and parameters [2]–[4]. Recent studies have applied quantum support vector classifiers, quantum convolutional networks and hybrid quantum neural networks to intrusion detection with promising results [5]–[8]. Current devices, however, belong to the noisy intermediate-scale quantum (NISQ) era [9]: circuits must stay shallow, qubit counts are small, and every circuit evaluation is expensive and noisy. A quantum detector has to get a lot out of each circuit call, and it has to tolerate some noise.

The other is neuromorphic computing. Spiking neural networks pass information as sparse binary events in time instead of dense real-valued activations [10]. Because a neuron only does work when it fires, SNNs can be far more energy efficient on neuromorphic hardware [11], [12], and surrogate gradient methods now let them be trained end to end with backpropagation [13], [14]. Since network traffic already arrives as a stream of events, spike-based processing suits it well.

In intrusion detection, the two directions have not yet met. Quantum IDS models usually treat each flow as a static feature vector, and spiking IDS models are fully classical. Early work on quantum spiking neurons [15], [16] suggests the two can be combined, but we found no study that builds and tests a quantum spiking network for network intrusion detection. That is what we do here. The main contributions are:

- We propose QSNN-NIDS, a hybrid network built from quantum leaky integrate-and-fire (QLIF) neurons, where a strongly entangling variational circuit supplies the input current and a classical membrane with learnable leak produces spikes over time.
- We evaluate it on two datasets, UNSW-NB15 and CIC-IDS2017, against three classical and two quantum baselines that use exactly the same features and splits.
- We measure how detection quality changes under simulated depolarizing noise at six levels, and show that both the noise tolerance and the way the model fails depend on the dataset.
- We report the quantum resource footprint, parameter count, spike sparsity and latency, and we check reproducibility with an independent re-run on a larger sample.

Section II reviews related work. Section III describes the datasets, the model, the baselines and the noise model. Section IV gives the experimental setup, Section V presents the results, Section VI covers limitations, and Section VII concludes.

II. Related Work

A. Machine Learning for Intrusion Detection

Classical IDS research has moved from signature matching to statistical and learned anomaly detection, using decision trees, ensembles, support vector machines and deep networks [1]. The choice of benchmark matters. Older sets such as KDD Cup 99 and NSL-KDD contain dated attacks and many duplicate records, which can inflate reported accuracy. UNSW-NB15 [17], [18] and CIC-IDS2017 [19] were built with modern traffic and attack types, and they are the two benchmarks we use.

B. Quantum Machine Learning for Intrusion Detection

Table I summarizes recent QML studies for intrusion detection. Kernel methods are the most common. Kavitha *et al.* [5] compared quantum support vector and quantum random forest classifiers on UNSW-NB15 for IoT traffic, Kalinin and Krundyshev [20] applied QSVM and QCNN to streaming security data, and Kumar and Swarnkar [7] built QuIDS around a quantum SVM that learns from few examples. Cirillo *et al.* [8], [21] benchmarked Pegasos-QSVC, variational classifiers and hybrid networks under hardware noise and found kernel methods to be the most robust. Singh *et al.* [22] reported that quantum kernels separate benign and malicious traffic more cleanly than classical models on CIC-IDS2017.

A second group uses circuit-based neural models. Gong *et al.* [6] proposed a variational quantum convolutional network for UNSW-NB15, Chao *et al.* [23] introduced a full-rotation QCNN, Amara *et al.* [24] designed a hybrid QCNN for healthcare IoT, and Wang *et al.* [25] showed that hybrid quantum convolutional models generalize well to unseen attacks on 5G-NIDD, although training became unstable when too many features were packed into a small circuit. Shaji *et al.* [26] compared quantum random forest and QCNN on ToN-IoT. For sequential traffic, Kadi *et al.* [27] and Do *et al.* [28] built quantum-enhanced LSTMs for DDoS detection and for managing the precision and recall trade-off. Other studies combined QML with blockchain logging [29], hybrid quantum neural networks for IoT [30], outlier analysis for DDoS [31], quantum metaheuristic optimization [32], software-defined networks [33] and noise-aware circuit mapping [34]. A broad survey of learned IDS for vehicular networks is given in [35].

TABLE I
Recent Quantum Machine Learning Studies for Intrusion Detection

Ref.	Model	Dataset	Reported result
[20]	QSVM, QCNN	Custom stream data	98.00%
[23]	Full-rotation QCNN	KDD Cup 99	99.20%
[6]	VQNN-QCNN	UNSW-NB15	94.51%
[31]	QIDS with outlier analysis	Custom DDoS traffic	99.90%
[5]	QSVC, QRF	UNSW-NB15	99.78%
[29]	QASB-IDS (CNN-LSTM, blockchain)	Edge-IIoTset, SWaT, X-IIoTID	99.10%
[25]	Hybrid QCNN variants	5G-NIDD	81.57% (unseen attacks)
[22]	QSVM vs. DNN, RF	CIC-IDS2017	Quantum above classical
[21]	Pegasos-QSVC, VQC, HQNN	ToN-IoT	93.05%
[24]	Hybrid QCNN	BoT-IoT (healthcare)	about 99%
[7]	QSVC (QuIDS)	Edge-IIoTset, ACI IoT	91.10% recall
[32]	QMGOA with DE	NSL-KDD, BoT-IoT, UNSW-NB15	99.89%
[34]	QuantumNetSec	UNSW-NB15, CIC-IDS2017, ToN-IoT	98.16%
[33]	HQNN, QSVM	UNR-IDD	78.24% (multi-class)
[26]	QRF, QCNN	ToN-IoT	95.00%
[30]	Hybrid QNN	BoT-IoT, ACI IoT, KDD	94.90%
[8]	Pegasos-QSVC	ToN-IoT, NSL-KDD	94.60%
[27]	QLSTM-E	CIC-DDoS2019	99.70%
[28]	Hybrid quantum-classical LSTM	CIC-IDS2017	97.43%
[35]	Survey of ML/DL methods	VeReMi, ToN-IoT, others	Survey
Ours	QSNN-NIDS (this work)	UNSW-NB15, CIC-IDS2017	86.30%, 97.46%

Results are as reported by each study and use different datasets and protocols, so they are not directly comparable.

C. Spiking and Quantum Spiking Neurons

The leaky integrate-and-fire neuron accumulates input current into a membrane potential that slowly leaks, and it emits a spike and resets when the potential crosses a threshold [10]. Because the spike function has no useful gradient, modern SNNs are trained with surrogate gradients that replace the derivative of the step function with a smooth curve [13], [14], [36]. Kristensen *et al.* [15] designed an artificial spiking quantum neuron built from a parameterized circuit, and Sun *et al.* [16] used quantum superposition ideas to encode information in spiking networks. Neither study addressed network security.

D. Research Gap

Looking across Table I, none of the quantum IDS models uses spiking dynamics over time. Each flow is a static vector, and every input costs a full circuit pass. Most papers report accuracy on a single dataset, often an older one, and few give false positive rates. Comparisons against other quantum models trained on the same inputs are rare, and so are noise studies and reports of quantum resource use (depth, entangling gates,

circuit calls). We designed QSNN-NIDS and its evaluation with these gaps in mind.

III. Methodology

Fig. 1 gives an overview of the pipeline. Raw flow records are cleaned and reduced classically, turned into spike trains, passed through two layers of QLIF neurons over T time steps, and read out as a benign or attack decision.

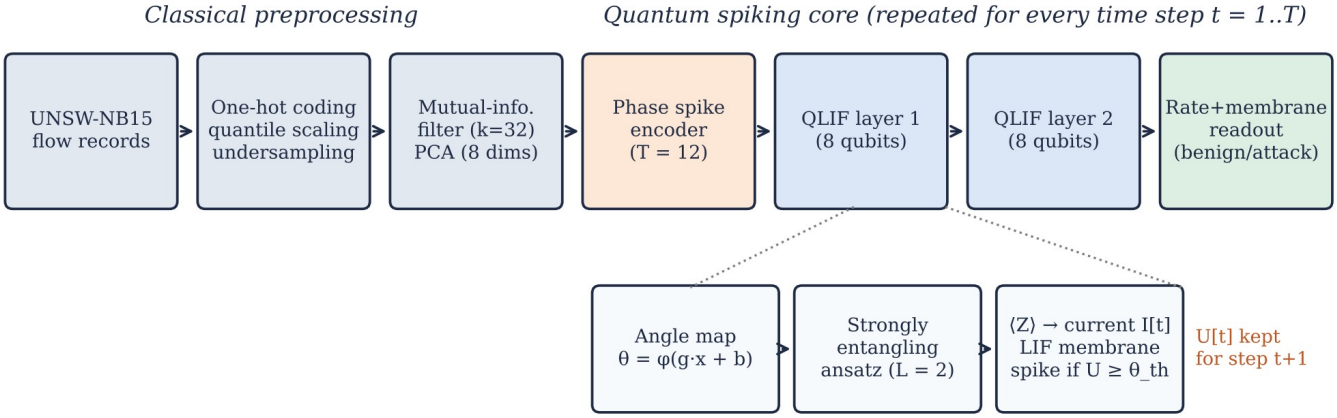


Fig. 1. Overview of QSNN-NIDS. Classical preprocessing reduces each flow to 8 features, which are phase-encoded into spikes and processed by two QLIF layers over T time steps. The lower row shows the computation inside one QLIF layer at a single time step.

A. Problem Formulation

Let each network flow be described by a feature vector **x** with a label y in {0, 1}, where 0 is benign and 1 is attack. The goal is to learn a classifier f(**x**) that predicts y. A missed attack and a false alarm do not cost the same, so we look at several metrics and not just accuracy.

B. Datasets and Preprocessing

UNSW-NB15 [17] was created at the Australian Centre for Cyber Security with the IXIA PerfectStorm tool. It mixes realistic normal traffic with nine attack families (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode and Worms), described by 49 flow, content, time and connection features. We drew 200,000 records from the combined official training and testing partitions. CIC-IDS2017 [19] was produced by the Canadian Institute for Cybersecurity and covers five days of traffic with brute force, DoS, DDoS, web, infiltration, botnet and port-scan attacks. We drew 300,000 flows from all eight release files. Both tasks are framed as binary detection.

The same pipeline was applied to both datasets, and every step was fitted on the training split only, then applied as is to the validation and test splits. Categorical fields were one-hot encoded and identifier and label columns were dropped. A quantile transformer mapped every feature to [0, 1], which tames the heavy-tailed byte and packet counts typical of flow data. The training split was balanced by random undersampling, while the validation and test splits keep their natural class mix. Mutual information with the label kept the 32 most informative features, and PCA reduced these to 8 components, one per qubit. Table II gives the resulting split sizes.

TABLE II
Dataset Samples and Splits

Item	UNSW-NB15	CIC-IDS2017
Source files	Official train + test	All 8 release files
Records drawn	200,000	300,000
Train (balanced)	88,138	73,156
Validation	13,014	26,437
Test	26,028	52,874
Features to model	8 (PCA)	8 (PCA)

C. Spike Encoding

Each 8-dimensional flow vector is converted into a binary spike tensor $S \in \{0,1\}^{T\times F}$ with T = 12 time steps and F = 8 input channels. We use phase coding, in which the scaled value of a feature controls where in the time window its spikes appear, rather than only how many there are. This carries more information per spike than simple rate coding. The encoder runs in stochastic mode during training, which acts as a light form of data augmentation.

$$s_i[t] = E(x_i, t) \in \{0, 1\}, \quad t = 1, \ldots, T \tag{1}$$

D. Quantum Leaky Integrate-and-Fire Neuron

The core of the model is the QLIF layer, which holds one neuron per qubit. At each time step the layer does four things.

1) Angle mapping. The incoming spike vector produces a drive $z_i[t]$ for each qubit, which is mapped to a bounded rotation angle by a scaled logistic function with gain g = 2 and bias b = −2:

$$\theta_i[t] = \varphi(g \cdot z_i[t] + b) \tag{2}$$

2) Variational circuit. The angles are loaded into 8 qubits by angle embedding, followed by a strongly entangling ansatz [3]

with L = 2 layers. Each layer applies a general single-qubit rotation Rot(α, β, γ) to every qubit, then a ring of CNOT gates whose range r grows with the layer index (Fig. 2). With three angles per qubit per layer, each QLIF layer holds 8 × 2 × 3 = 48 quantum weights **w**.

$$|\psi[t]\rangle = U_{ent}(\mathbf{w})\, U_{enc}(\boldsymbol{\theta}[t])\, |0\rangle^{\otimes 8} \tag{3}$$

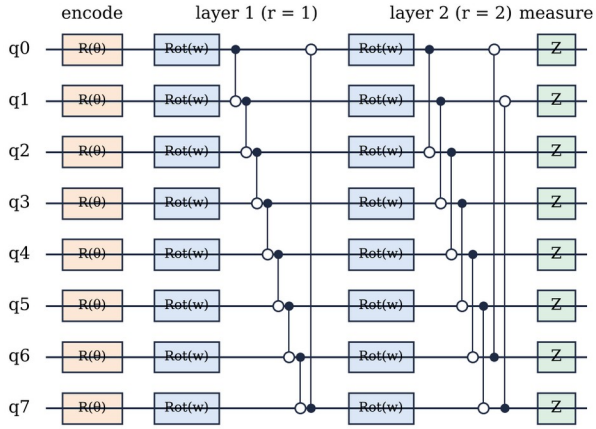


Fig. 2. Circuit used by one QLIF layer at each time step: angle embedding, two strongly entangling layers with CNOT range r = 1 and r = 2, and Pauli-Z measurement on all 8 qubits.

3) Measurement to current. Each qubit is measured in the Pauli-Z basis. The expectation value is converted into a non-negative input current, here written as the probability of observing the excited state:

$$I_i[t] = (1 - \langle\psi[t]|\, Z_i\, |\psi[t]\rangle) / 2 \tag{4}$$

4) Leaky membrane and spiking. The current is integrated into a membrane potential with leak factor β. A spike is emitted when the potential reaches the threshold V_{th}, and reset by subtraction keeps any excess charge:

$$U_i[t] = \beta\, U_i[t-1] + I_i[t] - S_i[t-1]\, V_{th} \tag{5}$$

$$S_i[t] = H(U_i[t] - V_{th}) \tag{6}$$

Here H is the Heaviside step function. We set $V_{th} = 0.55$ and initialize $\beta = 0.9$. The leak is learned through a logit parameter so that it stays between 0 and 1. The membrane gives each neuron a short memory, so the circuit at step t is influenced by everything the neuron has seen before.

E. Surrogate Gradient

The step function in (6) has zero gradient almost everywhere. During the backward pass we replace its derivative with the fast-sigmoid surrogate [36] with slope $\alpha = 5$:

$$\partial S / \partial U \approx 1 / (1 + \alpha\, |U - V_{th}|)^2 \tag{7}$$

Gradients flow through the surrogate into the membrane, through the measurement into the circuit, and into both the quantum weights and the learnable leak. On the simulator, gradients are computed exactly by backpropagation through the state vector.

F. Network Architecture and Readout

Fig. 3 shows the full network unrolled over time. Two QLIF layers are stacked, with dropout of 0.1 between them. The second layer runs on the spikes of the first. After T steps, the readout combines the output spike rate, the mean membrane potential and the mean measurement probability of each neuron, and a small linear layer maps these to two class logits:

$$r_i = (1/T) \textstyle\sum_t S_i[t], \quad \hat{\mathbf{y}} = \mathrm{softmax}(W_o\, [\mathbf{r}; \bar{\mathbf{u}}; \bar{\mathbf{p}}] + \mathbf{b}_o) \tag{8}$$

The full model has 340 trainable parameters: 96 quantum weights (two layers of 48) and 244 classical parameters in the drive, leak and readout. Each flow requires T × 2 = 24 circuit evaluations.

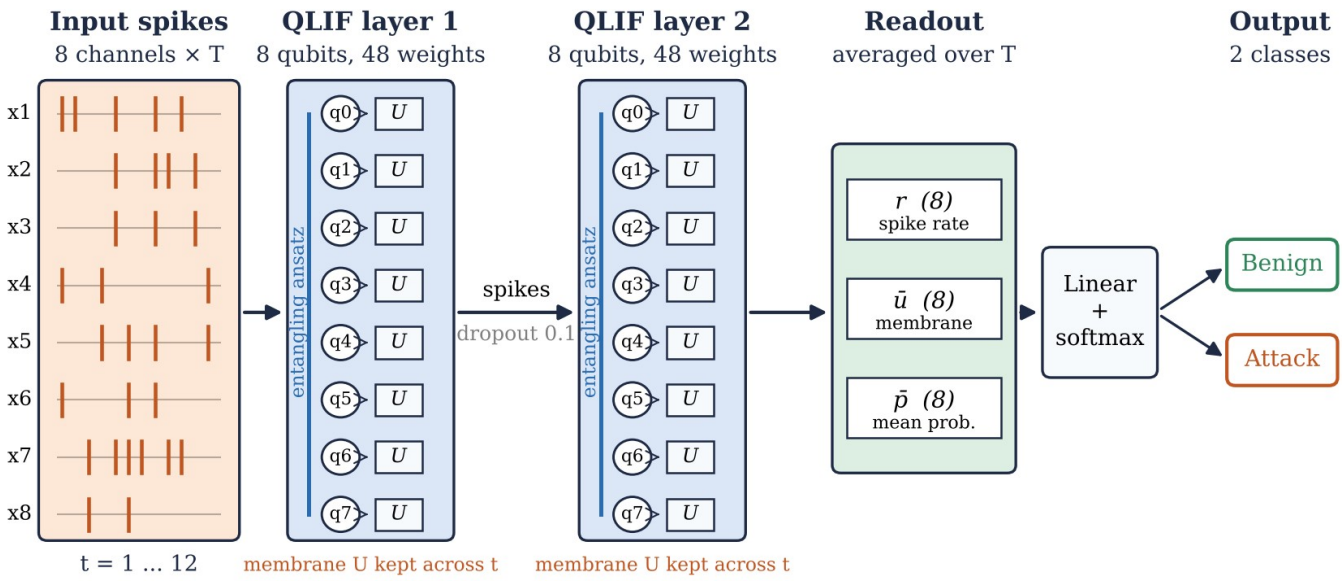


Fig. 3. Network architecture of QSNN-NIDS. Phase-coded input spikes drive two QLIF layers of 8 qubits each; every neuron keeps its membrane potential U from one time step to the next. After T = 12 steps, the readout averages the spike rate, membrane potential and measurement probability of each output neuron, and a linear layer with softmax gives the class.

G. Training Objective

We train with the focal loss [37], which down-weights easy examples so that training concentrates on flows near the decision boundary:

$$L_{FL} = -\alpha_c (1 - p_c)^{\gamma} \log(p_c) \quad (9)$$

Here p_c is the predicted probability of the true class and $\gamma = 2$. Because the training set is balanced, class weights α_c are both 1. Optimization uses AdamW [38] with separate learning rates for classical (3×10^{-3}) and quantum (10^{-2}) parameters, weight decay 10^{-4}, a cosine learning-rate schedule and gradient clipping at norm 1.0. Training stops early if validation F1 does not improve for 8 epochs, and the checkpoint with the best validation F1 is kept.

H. Baselines

All baselines are trained on exactly the same 8 PCA features and the same train, validation and test splits as QSNN-NIDS, so any gap in performance comes from the models and not from the inputs. The classical baselines are a Random Forest, an SVM with RBF kernel, and a multilayer perceptron (MLP). The quantum baselines are a quantum support vector classifier with a ZZ feature-map kernel (QSVC) [2] and a quantum convolutional neural network (QCNN) [39].

I. Noise Model

To test how the model behaves on noisy hardware, we evaluate the trained network with PennyLane's density-matrix simulator [40], adding a noise channel that combines depolarizing and amplitude-damping noise after every variational block. The depolarizing probability p is swept over six levels: 0, 0.005, 0.01, 0.02, 0.05 and 0.1. The model is not retrained under noise; this measures how a model trained on an ideal simulator holds up when its circuits run on imperfect qubits.

J. Evaluation Metrics

With attack as the positive class and TP, TN, FP and FN the confusion matrix counts, we report accuracy, precision, recall (detection rate), F1, false positive rate and, where available, the Matthews correlation coefficient:

$$\text{Precision} = TP / (TP + FP), \quad \text{Recall} = TP / (TP + FN) \quad (10)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \quad (11)$$

$$FPR = FP / (FP + TN) \quad (12)$$

$$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)} \quad (13)$$

We also report ROC-AUC. FPR matters most to operators, since every false positive is an alert someone must check. ROC-AUC is useful under noise because it shows whether the model's scores still rank attacks above benign flows even when a fixed decision threshold fails.

IV. Experimental Setup

All quantum circuits were simulated on CPU, with QSNN-NIDS implemented in PennyLane [40]. The ideal (noise-free) runs use exact state-vector simulation; the noise study uses the density-matrix simulator. A fixed random seed of 42 was used for data splitting, encoding and model initialization on both datasets. Table III lists the main settings, which are identical for both datasets.

Density-matrix simulation is much slower than state-vector simulation: one noise level on the full UNSW-NB15 test set took more than seven hours. The noise sweep was therefore run

on a stratified 2,000-flow subset of each test set, with the same class balance as the full test set, using the final trained weights of the model. Because of this, the noise-free (p = 0) row of the sweep differs slightly from the full test-set results in Tables IV and V.

This paper reports two runs on UNSW-NB15. The main results come from the full protocol described above. An earlier run, which we call the initial run, drew 50,000 records from the official training partition only and used a 7,284-flow test set; it is used in Section V-E as a reproducibility check. The two runs share the architecture, hyperparameters and seed, but not the data sample, so they are not two scales of the same experiment.

TABLE III
MODEL AND TRAINING SETTINGS

Setting	Value
Spike encoding, time steps T	Phase coding, 12
Qubits per layer, QLIF layers	8, 2
Ansatz, layers L	Strongly entangling, 2
Leak β (initial), learnable	0.9, yes
Threshold V_{th}, reset	0.55 (fixed), subtract
Angle map gain / bias	2.0 / −2.0
Surrogate, slope α	Fast sigmoid, 5.0
Readout, dropout	Rate + membrane + prob., 0.1
Loss	Focal, γ = 2
Optimizer, scheduler	AdamW, cosine
Learning rate (classical / quantum)	3×10^{-3} / 10^{-2}
Weight decay, grad. clip	10^{-4}, 1.0
Batch size, max epochs, patience	256, 30, 8
Seed	42

V. RESULTS AND DISCUSSION

A. Detection Performance on UNSW-NB15

Table IV compares all six models on the UNSW-NB15 test set of 26,028 flows. QSNN-NIDS reaches 86.30% accuracy, an F1 of 0.863, an MCC of 0.728 and a ROC-AUC of 0.938. Its recall of 0.890 means it catches about nine of every ten attacks, at a false positive rate of 16.3%.

TABLE IV
TEST RESULTS ON UNSW-NB15 (26,028 FLOWS, SAME FEATURES AND SPLIT FOR ALL MODELS)

Model	Acc.	Prec.	Rec.	F1	FPR	AUC
QSNN-NIDS	0.8630	0.8369	0.8903	0.8628	0.1626	0.9382
QSVC (ZZ)	0.8640	0.8417	0.8707	0.8559	0.1418	0.9361
QCNN	0.8749	0.8530	0.8957	0.8739	0.1446	0.9532
MLP (DNN)	0.8966	0.8826	0.9069	0.8946	0.1130	0.9715
RBF-SVM	0.8966	0.8812	0.9088	0.8948	0.1148	0.9662
Random Forest	**0.9048**	**0.8888**	**0.9180**	**0.9031**	**0.1076**	**0.9751**

QSNN-NIDS MCC = 0.7276. Best value in each column in bold.

Among the quantum models, QSNN-NIDS sits between the two baselines. Its F1 is higher than the QSVC's (0.863 against 0.856) and lower than the QCNN's (0.874), and its ROC-AUC follows the same order. Its false positive rate is the highest of the three (16.3% against 14.2% and 14.5%). All three classical baselines do better. Random Forest leads with 90.48% accuracy and an F1 of 0.903, which is 4.2 accuracy points and 0.040 F1 above QSNN-NIDS. The largest gap is in false positive rate, where Random Forest is 5.5 points lower.

B. Cross-Dataset Evaluation on CIC-IDS2017

To test whether the results hold beyond one benchmark, we repeated the full pipeline on CIC-IDS2017 with an identical architecture, hyperparameters and seed. Table V gives the results on the 52,874-flow test set, and Fig. 4 compares both datasets.

TABLE V
TEST RESULTS ON CIC-IDS2017 (52,874 FLOWS, SAME FEATURES AND SPLIT FOR ALL MODELS)

Model	Acc.	Prec.	Rec.	F1	FPR	AUC
QSNN-NIDS	0.9746	0.9187	0.9560	0.9370	0.0208	0.9939
QSVC (ZZ)	0.9720	0.9400	0.9216	0.9307	0.0151	0.9717
QCNN	0.9715	0.8967	0.9674	0.9307	0.0275	0.9953
MLP (DNN)	0.9910	0.9668	0.9882	0.9774	0.0084	0.9994
RBF-SVM	0.9901	0.9603	0.9910	0.9754	0.0101	0.9983
Random Forest	**0.9965**	**0.9877**	**0.9946**	**0.9912**	**0.0030**	**0.9997**

Best value in each column in bold.

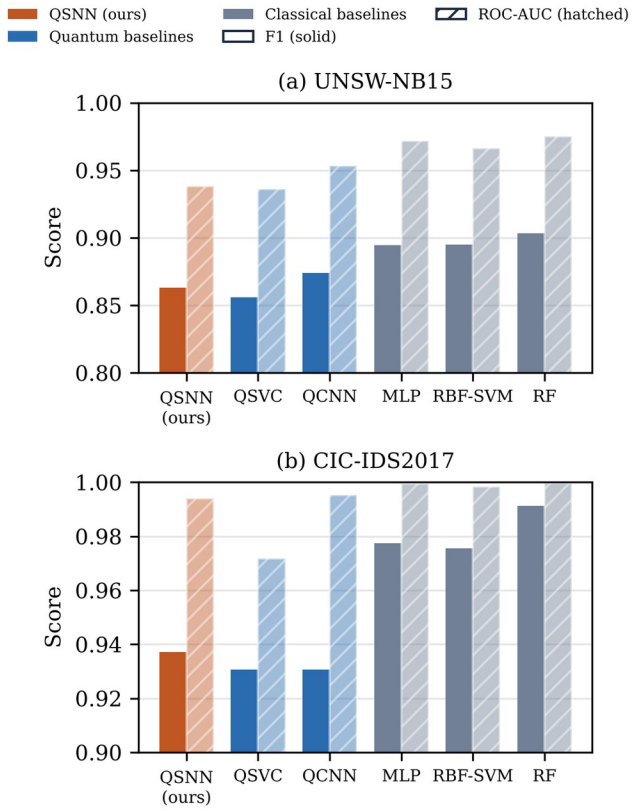


Fig. 4. Test-set F1 and ROC-AUC of QSNN-NIDS, the quantum baselines and the classical baselines on (a) UNSW-NB15 and (b) CIC-IDS2017. The two panels use different y-axis ranges.

QSNN-NIDS reaches 97.46% accuracy, an F1 of 0.937 and a ROC-AUC of 0.994, better than its own UNSW-NB15 numbers on every metric. Every other model also has higher accuracy and F1 on CIC-IDS2017, so this dataset is the easier one for this

feature set. Here QSNN-NIDS has the highest accuracy and F1 of the three quantum models, while the QCNN has a slightly higher ROC-AUC (0.995) and the QSVC a lower false positive rate (1.5% against 2.1%). Random Forest again leads overall, with 99.65% accuracy and an F1 of 0.991. The accuracy gap to Random Forest is smaller than on UNSW-NB15 (2.2 points against 4.2), while the F1 gap is larger (0.054 against 0.040).

The ordering of model families is the same on both datasets: classical baselines first, with QSNN-NIDS level with the quantum baselines. The architecture did not need to be changed to move from one dataset to the other.

C. Noise Robustness

Table VI and Fig. 5 show how detection quality changes as depolarizing noise increases. On both datasets the model is stable through p = 0.02. On UNSW-NB15 it keeps 98.3%, 98.1% and 97.1% of its noise-free F1 at p = 0.005, 0.01 and 0.02. On CIC-IDS2017 it keeps 100.7%, 99.8% and 99.4%; the small gain at p = 0.005 is likely sampling variation on a 2,000-flow subset.

TABLE VI
QSNN-NIDS UNDER DEPOLARIZING NOISE (2,000-FLOW STRATIFIED TEST SUBSETS)

p	UNSW Acc.	UNSW F1	UNSW AUC	CIC Acc.	CIC F1	CIC AUC
0	0.8575	0.8577	0.9331	0.9690	0.9235	0.9917
0.005	0.8485	0.8429	0.9321	0.9715	0.9295	0.9923
0.01	0.8455	0.8413	0.9318	0.9680	0.9212	0.9917
0.02	0.8430	0.8328	0.9286	0.9680	0.9179	0.9910
0.05	0.5165	0.0000	0.9133	0.8345	0.2789	0.9809
0.1	0.5165	0.0000	0.8594	0.8025	0.0000	0.4713

F1 retention relative to p = 0, UNSW-NB15: 98.3%, 98.1%, 97.1%, 0%, 0%. CIC-IDS2017: 100.7%, 99.8%, 99.4%, 30.2%, 0%.

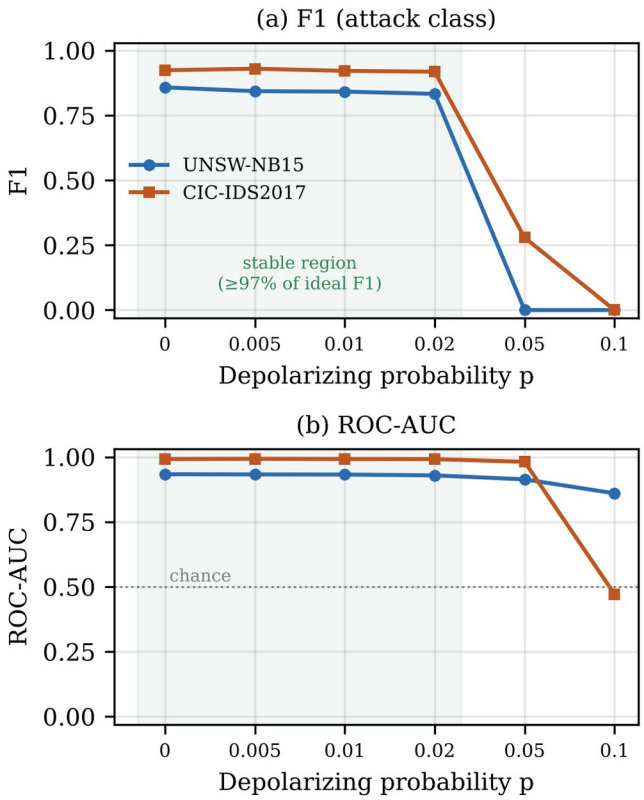


Fig. 5. QSNN-NIDS under depolarizing noise. (a) F1 of the attack class. (b) ROC-AUC. The shaded region is where the model keeps at least 97% of its noise-free F1 on both datasets.

Past p = 0.02 the two datasets fail in different ways. On UNSW-NB15 the model collapses at p = 0.05: it labels every flow as benign, so accuracy falls to 0.5165, which is exactly the share of benign flows in the subset, and F1 drops to zero. ROC-AUC, however, stays at 0.913 at p = 0.05 and 0.859 at p = 0.1. The scores still rank attacks above benign traffic; what breaks is the fixed decision threshold, because noise pulls every output toward the same side of it. On CIC-IDS2017 the decline at p = 0.05 is more gradual (F1 0.279, ROC-AUC 0.981), but at p = 0.1 both measures fail together and ROC-AUC drops to 0.471, which is no better than chance. In this case the signal itself is lost, not only the threshold.

Two practical points follow. First, noise tolerance should be measured for each deployment rather than assumed to carry over from another dataset. Second, the UNSW-NB15 result suggests that recalibrating the threshold under noise could recover much of the lost F1, since the ranking survives. We did not test recalibration here.

D. Resource Use and Efficiency

Table VII lists the cost of QSNN-NIDS on both datasets. The architecture is the same, so the qubit count, circuit depth and parameter count do not change; only the learned weights, the spike rate and the latency differ.

TABLE VII
QUANTUM RESOURCE FOOTPRINT AND EFFICIENCY OF QSNN-NIDS

Quantity	UNSW-NB15	CIC-IDS2017
Qubits per QLIF layer	8	8
Circuit depth per time step	15	15
Total circuit depth (T = 12)	360	360
Two-qubit gates per time step	32	32
Circuit evaluations per flow	24	24
Quantum / classical parameters	96 / 244	96 / 244
Mean spike rate	14.4%	18.0%
Spike events per flow (of 192)	about 28	about 35
Sparsity saving	85.6%	82.0%
Latency per flow (CPU sim.)	7.91 ms	5.10 ms
Throughput (CPU sim.)	126 flows/s	196 flows/s

Each QLIF layer uses 8 qubits and a circuit of depth 15 per time step. One time step uses 32 two-qubit gates across both layers, so a full 12-step pass over one flow uses 384 two-qubit gates spread over 24 circuit evaluations of 8 qubits each. Circuits of this width and depth are within the range that current superconducting and trapped-ion devices can run, and the model kept at least 97% of its F1 at a depolarizing rate of 0.02.

The cost of a QSNN-NIDS prediction is fixed at 24 circuit evaluations per flow, whatever the size of the training set. A kernel model such as the QSVC needs one kernel circuit for every pairing of the new flow with a stored support vector, so its inference cost grows with the number of support vectors. The QCNN, on the other hand, needs fewer circuit calls per flow than QSNN-NIDS, because it does not unroll over time. The QSNN design trades extra circuit calls for temporal state and sparse output.

Spiking makes the network's output sparse. With 16 neurons and 12 time steps, a flow has 192 possible spike events. At the measured mean spike rates of 14.4% on UNSW-NB15 and 18.0% on CIC-IDS2017, about 28 and 35 of these occur, so 85.6% and 82.0% of potential events are skipped. On neuromorphic hardware a silent neuron does no downstream work, so this sparsity translates directly into fewer operations. We did not measure energy, so we report the event counts rather than an energy figure.

With 340 parameters in total, 96 of them quantum, the trained model fits in a checkpoint of under 9 KB. Inference on the CPU simulator took 7.91 ms per flow on UNSW-NB15 and 5.10 ms on CIC-IDS2017 (126 and 196 flows per second). These figures are the cost of simulating quantum states on a classical machine. They say nothing about the speed of real quantum hardware and are given only as a reference point.

E. Reproducibility Check

Table VIII compares the initial UNSW-NB15 run with the full-protocol run. The two runs differ in data: the initial run used 50,000 records from the official training partition only, while the full protocol drew 200,000 records from both official partitions, after we found that the loader had been silently skipping the official testing file. Architecture, hyperparameters and seed are the same.

TABLE VIII
REPRODUCIBILITY CHECK ON UNSW-NB15: INITIAL RUN VS. FULL PROTOCOL

Metric	Initial run	Full protocol	Change
Test flows	7,284	26,028	
Accuracy	0.8585	0.8630	+0.0045
Precision (attack)	0.7904	0.8369	+0.0465
Recall (attack)	0.8981	0.8903	−0.0078
F1 (attack)	0.8408	0.8628	+0.0220
FPR	0.1698	0.1626	−0.0072
MCC	0.7190	0.7276	+0.0086
ROC-AUC	0.9397	0.9382	−0.0015
Mean spike rate	0.284	0.144	−0.140
RF accuracy (baseline)	0.9334	0.9048	−0.0286

Initial run: 50,000 records from the official training partition only. Full protocol: 200,000 records from both official partitions. Same architecture, hyperparameters and seed.

Every detection metric agrees to within 0.05, and accuracy, recall, FPR, MCC and ROC-AUC agree to within 0.01. The largest change is in attack precision, which rose from 0.790 to 0.837 on the larger and more varied sample. Two other differences are worth stating. The mean spike rate halved, from 28.4% to 14.4%, while detection quality stayed the same, so the model reached similar accuracy with much sparser firing on more data. The Random Forest baseline also moved, from 93.34% to 90.48% accuracy; Random Forest does not use the spike encoder, so this change comes from the different test distribution and not from the QSNN pipeline. Latency doubled (3.55 to 3.79 ms against 7.91 ms), but the two runs used different machines and software builds, so the latencies are not directly comparable. Fig. 6 shows the confusion matrix of the initial run.

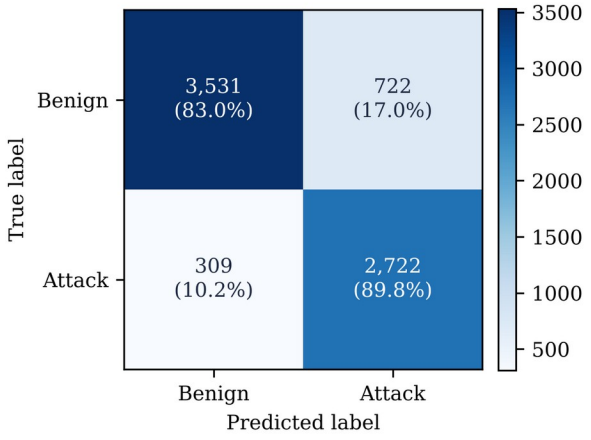


Fig. 6. Confusion matrix of the initial UNSW-NB15 run (7,284 test flows). Row percentages are shown in brackets.

F. Discussion

Quantum spiking computation can be trained end to end for intrusion detection, and the result carries across two datasets and two independent runs. Against other quantum models

trained on the same inputs, QSNN-NIDS is competitive: it has the best F1 of the three on CIC-IDS2017 and lies between the QSVC and QCNN on UNSW-NB15. Against classical models it is not yet competitive, and Random Forest remains the better detector on both datasets.

The case for the model therefore rests on its cost profile rather than its accuracy: a fixed 24 circuit calls per flow on 8 qubits, 340 parameters, 82% to 86% of spike events skipped, and stable behaviour up to a depolarizing rate of 0.02. Whether this translates into a real advantage depends on measurements we have not yet made, mainly energy on neuromorphic hardware and accuracy on real quantum devices.

VI. Limitations and Threats to Validity

All results are from a single seed (42) and a single split per dataset. Multi-seed runs to report means and standard deviations were started but not completed in the available time, so we cannot give confidence intervals, and differences of one or two points between models should be read with this in mind. A run on real IBM quantum hardware (the ibm_fez backend) was prepared but could not be completed because the account's allotted hardware time had been used up; all noise results are simulated. The noise sweep used a 2,000-flow stratified subset of each test set rather than the full test set. We did not measure energy use, and latencies are CPU simulator figures. Finally, the task is binary detection; we did not evaluate per-family detection, where rare UNSW-NB15 classes such as Worms have very few samples and need a different sampling design.

VII. Conclusion and Future Work

We presented QSNN-NIDS, a quantum spiking neural network for network intrusion detection built from quantum leaky integrate-and-fire neurons. Each neuron uses an 8-qubit strongly entangling circuit to drive a leaky membrane, and the network processes phase-coded flow features over 12 time steps. On UNSW-NB15 the model reached an F1 of 0.863 and ROC-AUC of 0.938, and on CIC-IDS2017 an F1 of 0.937 and ROC-AUC of 0.994, with 340 parameters and 82% to 86% spike sparsity. It matched the quantum baselines, trailed the classical ones on both datasets, and kept at least 97% of its F1 under depolarizing noise up to $p = 0.02$. An independent re-run reproduced the initial results.

The most pressing next steps are to complete the multi-seed runs and report confidence intervals, and to run the model on real quantum hardware. After that, we plan to study threshold recalibration under noise and per-family attack detection.

Code Availability

The initial experiment, including preprocessing, training logs, configuration and results, is available as a public Kaggle notebook at https://www.kaggle.com/code/hiemalrana22/qml-notebook.

Acknowledgment

The authors used an AI assistant to help draft and edit the text of this manuscript. The model, code, experiments and results are the authors' own, and the authors reviewed and take responsibility for all content.

References

[1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, Art. no. 20, 2019.

[2] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019.

[3] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, "Circuit-centric quantum classifiers," *Phys. Rev. A*, vol. 101, no. 3, Art. no. 032308, 2020.

[4] M. Cerezo *et al.*, "Variational quantum algorithms," *Nat. Rev. Phys.*, vol. 3, pp. 625–644, 2021.

[5] S. S. Kavitha, M. D. Dhanush Sagar, and S. Deepak, "Internet of Things network intrusion detection system using quantum and classical machine learning," *IET Quantum Commun.*, vol. 6, no. 1, Art. no. e70020, 2025, doi: 10.1049/qtc2.70020.

[6] C. Gong, W. Guan, H. Zhu, A. Gani, and H. Qi, "Network intrusion detection based on variational quantum convolution neural network," *J. Supercomput.*, vol. 80, no. 9, pp. 12743–12770, 2024, doi: 10.1007/s11227-024-05919-y.

[7] R. Kumar and M. Swarnkar, "QuIDS: A quantum support vector machine-based intrusion detection system for IoT networks," *J. Netw. Comput. Appl.*, vol. 234, Art. no. 104072, 2025, doi: 10.1016/j.jnca.2024.104072.

[8] F. Cirillo, C. Esposito, and J. T. Seo, "Benchmarking quantum machine learning methods for intrusion detection on noisy quantum computers," *Quantum Mach. Intell.*, vol. 8, no. 1, Art. no. 27, 2026, doi: 10.1007/s42484-026-00379-4.

[9] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[10] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Netw.*, vol. 10, no. 9, pp. 1659–1671, 1997.

[11] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, pp. 607–617, 2019.

[12] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[13] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, 2019.

[14] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proc. IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[15] L. B. Kristensen, M. Degroote, P. Wittek, A. Aspuru-Guzik, and N. T. Zinner, "An artificial spiking quantum neuron," *npj Quantum Inf.*, vol. 7, Art. no. 59, 2021.

[16] Y. Sun, Y. Zeng, and T. Zhang, "Quantum superposition inspired spiking neural network," *iScience*, vol. 24, no. 8, Art. no. 102880, 2021.

[17] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.

[18] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Secur. J.: A Global Perspective*, vol. 25, no. 1–3, pp. 18–31, 2016.

[19] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.

[20] M. Kalinin and V. Krundyshev, "Security intrusion detection using quantum machine learning techniques," *J. Comput. Virol. Hacking Tech.*, vol. 19, no. 1, pp. 125–136, 2023, doi: 10.1007/s11416-022-00435-0.

[21] F. Cirillo and C. Esposito, "Quantum machine learning for intrusion detection on noisy quantum computers," in *Proc. IEEE Int. Conf.*

Quantum Comput. Eng. (QCE), 2025, pp. 1665–1675, doi: 10.1109/QCE65121.2025.00184.

[22] B. Singh, S. Indu, and S. Majumdar, "Comparative analysis of intrusion detection models using quantum machine learning techniques," *Circuits Syst. Signal Process.*, vol. 44, no. 12, pp. 9592–9618, 2025, doi: 10.1007/s00034-025-03256-w.

[23] S. Chao, G. Yang, M. Nie, Y. Liu, and M. Zhang, "Full-rotation quantum convolutional neural network for abnormal intrusion detection system," in *Proc. 5th Int. Conf. Artif. Intell. Pattern Recognit. (AIPR)*, 2022, pp. 852–859, doi: 10.1145/3573942.3574105.

[24] M. Amara, M. Amara, S. Mnasri, and T. Val, "QCNN-ID: A quantum-classical hybrid model for IoT intrusion detection," *Procedia Comput. Sci.*, vol. 270, pp. 2196–2204, 2025, doi: 10.1016/j.procs.2025.09.340.

[25] Z. Wang, K. W. Fok, and V. L. L. Thing, "Network attack traffic detection with hybrid quantum-enhanced convolution neural network," *Quantum Mach. Intell.*, vol. 7, no. 1, Art. no. 50, 2025, doi: 10.1007/s42484-025-00278-0.

[26] A. A. Shaji, S. Sadhwani, R. Muthalagu, P. M. Pawar, and T. Mathew, "QML-IDS: Quantum machine learning approaches for intrusion detection systems in IoT devices," in *Proc. 3rd Int. Conf. Comput. Intell. Netw. Syst. (CINS)*, 2025, pp. 1–7, doi: 10.1109/CINS67018.2025.11412140.

[27] A. Kadi, H. Moudoud, L. Khoukhi, and Z. Abou El Houda, "Quantum-enhanced LSTM for sequential network flow analysis: A hybrid approach to DDoS detection," in *Proc. Int. Conf. Quantum Commun., Netw., Comput. (QCNC)*, 2026, pp. 830–834, doi: 10.1109/QCNC69040.2026.00136.

[28] P. H. Do, T. D. Le, T. D. Dinh, and V.-D. Pham, "Navigating the precision-recall trade-off in network intrusion detection with a hybrid quantum-classical LSTM," in *Proc. 40th Int. Conf. Inf. Netw. (ICOIN)*, 2026, pp. 1006–1011, doi: 10.1109/ICOIN68469.2026.11480537.

[29] N. Hussain, S. Li, A. Hussain, Z. Ullah, and M. Jamjoom, "Quantum-aware secure blockchain intrusion detection system for industrial IoT networks," *Sci. Rep.*, vol. 16, Art. no. 2265, 2026, doi: 10.1038/s41598-025-31985-0.

[30] I. Bharathi, V. Sonai, and S. Sridevi, "Quantum-driven enhanced machine learning algorithm for intrusion detection in Internet of things environment," *EPJ Quantum Technol.*, vol. 13, no. 1, Art. no. 20, 2026, doi: 10.1140/epjqt/s40507-026-00463-5.

[31] T. H. Kim and S. Madhavi, "Quantum intrusion detection system using outlier analysis," *Sci. Rep.*, vol. 14, Art. no. 27114, 2024, doi: 10.1038/s41598-024-78389-0.

[32] A. K. Shukla, "Safeguarding networks from malicious intrusions using quantum machine intelligence," *Softw. Pract. Exp.*, vol. 55, no. 10, pp. 1676–1695, 2025, doi: 10.1002/spe.70004.

[33] J. Lo Anguera *et al.*, "Leveraging quantum machine learning for intrusion detection in software-defined networks," in *Proc. IEEE Int. Conf. Mach. Learn. Commun. Netw. (ICMLCN)*, 2025, pp. 1–6, doi: 10.1109/ICMLCN64995.2025.11140473.

[34] D. Abreu, D. Moura, C. E. Rothenberg, and A. Abelém, "QuantumNetSec: Quantum machine learning for network security," *Int. J. Netw. Manag.*, vol. 35, no. 4, Art. no. e70018, 2025, doi: 10.1002/nem.70018.

[35] I. Chouchene, H. Eltaief, and H. Youssef, "Advances in machine and deep learning for intrusion detection in VANETs: a comprehensive survey," *Cluster Comput.*, vol. 29, no. 3, Art. no. 143, 2026, doi: 10.1007/s10586-026-05987-0.

[36] F. Zenke and S. Ganguli, "SuperSpike: Supervised learning in multilayer spiking neural networks," *Neural Comput.*, vol. 30, no. 6, pp. 1514–1541, 2018.

[37] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.

[38] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[39] I. Cong, S. Choi, and M. D. Lukin, "Quantum convolutional neural networks," *Nat. Phys.*, vol. 15, pp. 1273–1278, 2019.

[40] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," arXiv:1811.04968, 2018.